

From Zero to a Live Website

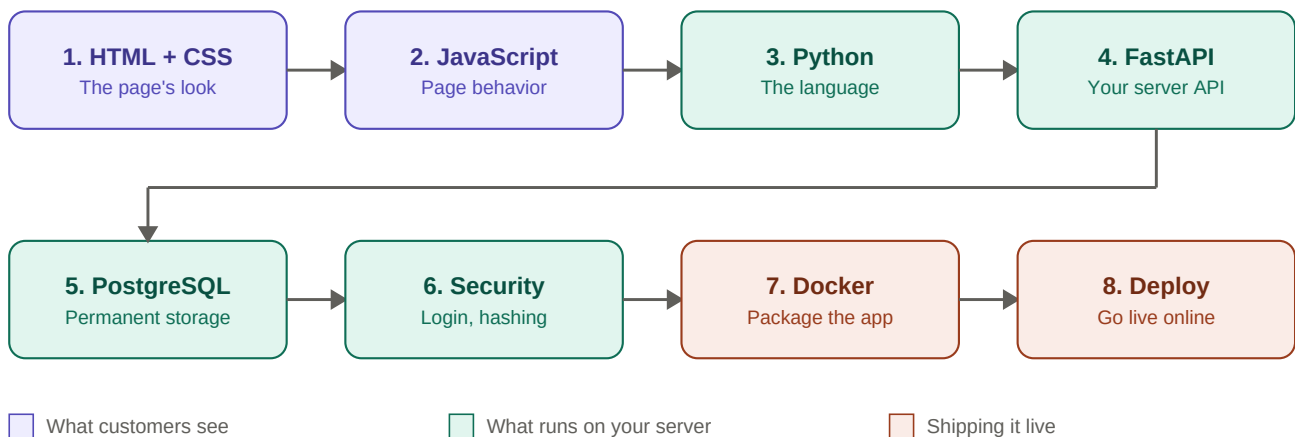
A step-by-step learning roadmap with FastAPI, from your first HTML file to a deployed, secure site that stores company data, followed by how real companies build theirs.

How to read this guide. Part 1 lists every technology in the order you should learn it. Each one answers four questions: what it is, why you need it, where it is used, and what to learn. Part 2 takes real companies and explains their stacks the same way, then compares them with yours.

Part 1 - The roadmap

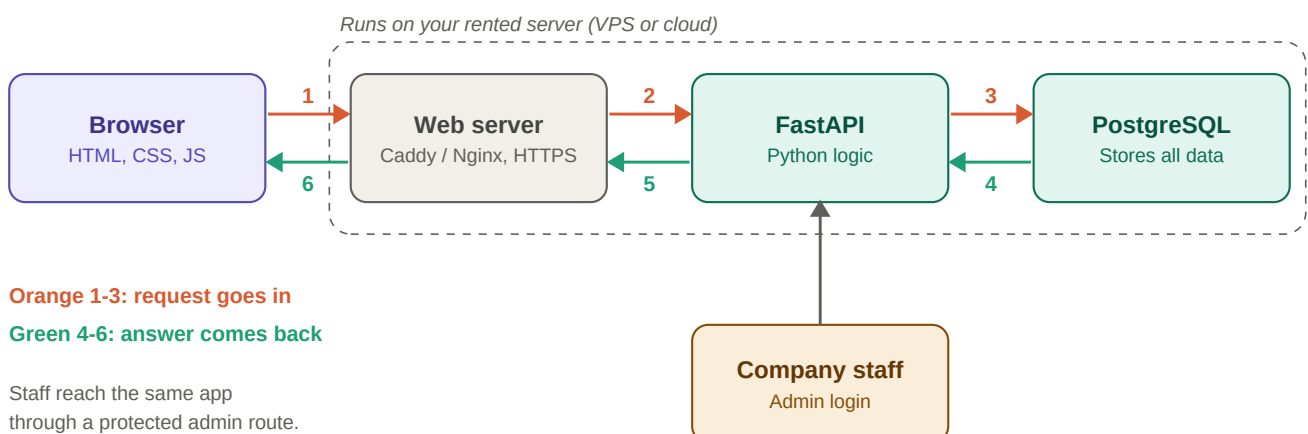
The eight stages at a glance

Git (version control) runs alongside every stage.



What happens when a customer uses your live site

Once everything is deployed, this is the path every action takes. Learn the stack so that you understand each box in this picture.



Example: a customer submits the contact form

- The customer types your domain. DNS (the internet's phone book) points their browser to your server's address.
- The browser connects over HTTPS, so everything is encrypted, and downloads your HTML, CSS and JavaScript.
- The customer fills in the form and clicks Send. Your JavaScript calls `fetch("/contact", ...)` (arrow 1).
- The web server (Caddy or Nginx) forwards the request to FastAPI (arrow 2).
- FastAPI checks the data with Pydantic, applies rate limiting, and saves it to PostgreSQL (arrow 3).
- The database confirms, FastAPI replies `{"status": "received"}`, and JavaScript shows "Thank you" (arrows 4 to 6).
- Later, a staff member logs in. FastAPI checks their password hash and role, then lists the saved messages from the database.

The steps, in order

Step 0 - Git and the terminal (day 1, then every day)

What it is	Git saves versions of your code. The terminal is the text window where you run commands. GitHub stores your Git history online.
Why you need it	You can undo mistakes, work on features safely, back up your code, and later deploy automatically from GitHub.
Where it is used	Every stage below. Commit after every small change.
What to learn	git init, add, commit, branch, merge, push; basic terminal navigation (cd, ls, mkdir).

Step 1 - HTML (a few days)

What it is	The language that describes the content of a page: headings, text, images, links, forms, buttons.
Why you need it	A browser only understands HTML as "a page". Everything customers see is HTML in the end.
Where it is used	Every page: home, contact form, login form, admin table.
What to learn	Semantic tags (header, main, section, footer), links, images, forms and inputs, accessibility basics.

Step 2 - CSS (1 to 2 weeks)

What it is	The language that controls how HTML looks: colors, fonts, spacing, layout.
Why you need it	HTML alone looks like a plain document. CSS makes the site look professional and work on phones.
Where it is used	All styling of every page, including the mobile layout.
What to learn	Box model, Flexbox, Grid, media queries (responsive design), CSS variables.
Skip for now	Bootstrap, Tailwind and Sass. Learn plain CSS first; frameworks make sense afterwards.

Step 3 - JavaScript (3 to 4 weeks)

What it is	The only programming language that runs inside the browser.
Why you need it	HTML and CSS are static. JavaScript reacts when a user clicks, validates a form before sending it, and talks to your server in the background.
Where it is used	The Send button on the contact form, showing a success message, loading the admin message list without refreshing.
What to learn	Variables, functions, arrays and objects, DOM manipulation, events, fetch() with async/await, JSON.
Skip for now	React, Vue and TypeScript. Add them later only if the site becomes complex.

Step 4 - Python (2 to 3 weeks)

What it is	A readable general-purpose language. This is the language FastAPI is written in.
Why you need it	Your server logic (who can log in, what gets saved, what is returned) is Python code.
Where it is used	Every endpoint of your server.
What to learn	Functions, dictionaries and lists, classes, type hints, modules, virtual environments (venv), pip, reading error messages.

Step 5 - FastAPI (3 to 4 weeks)

What it is	A Python framework for building a web server. It receives a request from the browser, runs your code, and sends a response.
Why you need it	The browser must never talk to the database directly. FastAPI sits between them and decides what is allowed. It also gives you automatic input checking (Pydantic) and automatic interactive docs at /docs.
Where it is used	Sign up, log in, save a contact message, list orders for the company, send emails.
What to learn	Routes (GET, POST, PUT, DELETE), path and query parameters, Pydantic models, dependencies (Depends), error handling, routers to split code into files, optional Jinja2 templates to serve HTML pages.

```
from fastapi import FastAPI
from pydantic import BaseModel, EmailStr

app = FastAPI()

class Contact(BaseModel):
    name: str
    email: EmailStr
    message: str

@app.post("/contact")
def save_contact(data: Contact):
    # later: save to PostgreSQL
    return {"status": "received"}
```

Step 6 - PostgreSQL with SQLAlchemy and Alembic (3 weeks)

What it is	PostgreSQL is a database: organized tables on disk. SQLAlchemy lets you read and write it using Python objects. Alembic records every change you make to the table structure.
Why you need it	Variables in Python vanish when the server restarts. A database keeps data permanently. This is where the company's data lives.
Where it is used	Users, contact messages, orders, anything that must survive a restart.
What to learn	SQL basics (SELECT, INSERT, UPDATE, DELETE, JOIN), primary and foreign keys, indexes, SQLAlchemy models and sessions, Alembic migrations.

Step 7 - Security (1 to 2 weeks, but think about it from day one)

What it is	A set of habits and tools that keep customers' data safe.
Why you need it	A website that stores real people's data is a target the moment it goes online.
Where it is used	Everywhere. Your site is only as safe as its weakest page.
What to learn	See the checklist on the next page.

Security checklist

Risk	What to do	Tool or idea
Stolen passwords	Never store passwords as plain text. Store a one-way hash.	Argon2 or bcrypt (for example the <code>pwdlib</code> or <code>argon2-cffi</code> package)
Wrong person sees data	Check who the user is, then what they are allowed to do.	JWT tokens or session cookies; roles such as customer and admin
Leaked secrets	Keep database passwords and keys outside your code. Never commit them.	Environment variables, <code>.env</code> file in <code>.gitignore</code>
Bad or malicious input	Validate everything that comes in.	Pydantic models; use the ORM, never build SQL by joining strings (prevents SQL injection)
Script injected into pages	Escape user text before showing it in HTML (XSS).	Jinja2 auto-escaping; avoid <code>innerHTML</code> with user data
Another site misusing your API	Allow only your own domain to call the API from browsers.	CORS settings; CSRF protection if you use cookies
Password guessing and spam	Limit how often one visitor can try.	Rate limiting (for example the <code>slowapi</code> package)
Traffic read on the network	Encrypt all traffic.	HTTPS via Let's Encrypt (Caddy does this automatically)
Old vulnerable libraries	Update dependencies regularly.	<code>pip list --outdated</code> , GitHub Dependabot alerts

```
from fastapi import Depends, HTTPException

def require_admin(user = Depends(get_current_user)):
    if user.role != "admin":
        raise HTTPException(status_code=403, detail="Admins only")
    return user

@app.get("/admin/messages")
def list_messages(admin = Depends(require_admin)):
    ... # read messages from PostgreSQL
```

Step 8 - Testing with pytest (1 week)

What it is	Small programs that call your endpoints and check the answers.
Why you need it	After every change you learn within seconds whether something else broke.
Where it is used	Before every deploy. FastAPI ships a <code>TestClient</code> for this.

Step 9 - Docker (1 week)

What it is	Packages Python, your code and its libraries into a container that runs identically anywhere.
Why you need it	"It works on my laptop" stops being a problem. The server runs exactly what you tested.
Where it is used	You run FastAPI, PostgreSQL and the web server as separate containers on one server, started together by Docker Compose.
What to learn	Dockerfile, images and containers, volumes (so database files survive), Docker Compose, environment files.

```
# Dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# docker-compose.yml
services:
  app:
    build: .
    env_file: .env
    depends_on: [db]
  db:
    image: postgres:16
    env_file: .env
    volumes: [pgdata:/var/lib/postgresql/data]
  caddy:
    image: caddy:2
    ports: ["80:80", "443:443"]
    volumes: [".:/Caddyfile:/etc/caddy/Caddyfile", "caddy_data:/data"]
volumes:
  pgdata:
  caddy_data:

# Caddyfile (Caddy fetches the HTTPS certificate by itself)
yourdomain.com {
  reverse_proxy app:8000
}
```

Step 10 - Deployment and operations (1 to 2 weeks)

What it is	Putting your containers on a computer that is always on and reachable from the internet.
Why you need it	This is the step that turns "works on my laptop" into "customers can open it".
Where it is used	Your live site.
What to learn	Rent a small cloud server (a VPS) or use a platform such as Render or Railway; buy a domain and point it to the server with DNS records; use SSH to log in; set up a firewall (only ports 80, 443 and SSH open); set up automatic database backups (and test restoring one); read logs; add an uptime monitor.
Optional later	GitHub Actions to test and deploy automatically on every push.

What a minimal but complete website includes

Feature	Technology that covers it
Pages and design	HTML, CSS, JavaScript
Works on phones	CSS media queries, Flexbox, Grid
Contact form or signup	FastAPI endpoint with Pydantic validation
Data storage	PostgreSQL
Customer login	Password hashing and JWT or session cookies
Admin area for the company	Role-based routes (or the SQLAlchemy library)
Emails (thank-you, password reset)	SMTP or an email service (Resend, SendGrid)
Secure traffic	HTTPS with Let's Encrypt via Caddy or Nginx
Secrets kept safe	Environment variables, never committed to Git
Protection against abuse	Rate limiting, input validation, CORS rules
Online all the time	Docker, a cloud server, a domain
Not losing data	Automatic database backups
Knowing when it breaks	Logs and an uptime monitor

Suggested timeline

Weeks	Focus	What you should have at the end
1 - 4	Git, HTML, CSS	A responsive static page, put online early (even on a free host)
5 - 8	JavaScript	A page with a working form that validates input
9 - 11	Python	Small scripts; comfortable with errors and packages
12 - 15	FastAPI	An API with a contact endpoint and automatic docs
16 - 18	PostgreSQL	Messages saved permanently; migrations working
19 - 20	Security and tests	Signup, login, admin-only route, pytest suite
21 - 24	Docker and deployment	The whole project live on your domain with HTTPS and backups

That is roughly 4 to 6 months at 2 to 3 hours a day. Do not wait until the end to deploy. Deployment is the hardest part for beginners, and repeating it makes it easy.

Practice project

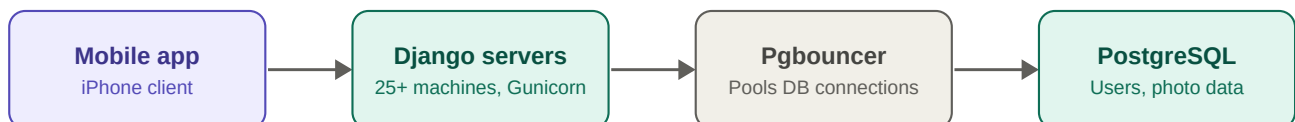
Build a small business website with a home page, a contact form, customer signup and login, and an admin page listing every saved message. It touches every technology above. If you can finish it, you can build any minimal site.

Part 2 - How real companies do it

Read this honestly. Companies rarely publish their complete stack. The pictures below are public snapshots from the companies' own engineering posts and documentation, and some are several years old (Instagram 2011 to 2012, Stack Overflow 2013 to 2016). Stacks change. Treat them as examples of *choices and reasons*, not as today's exact setup. Where I did not verify a layer, I say so.

Example A - Instagram, early years (Python, Django, PostgreSQL)

The closest real-world match to the path you are taking: a Python web framework plus PostgreSQL. In its first year Instagram grew to 14 million users with a team of three engineers, running on Amazon's cloud.



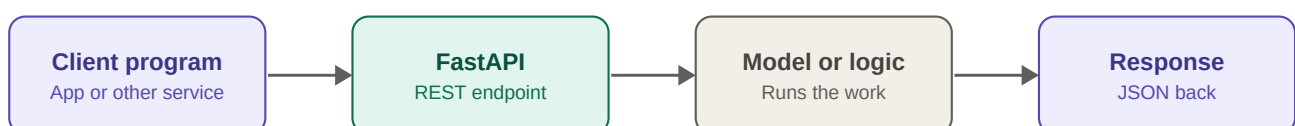
Redis and Memcached sit beside Django as fast caches. Sentry watches for errors.

Question	Instagram's answer
What the customer sees	A phone app rather than a website, so the browser stage (HTML, CSS, JavaScript) was replaced by the iOS app. The server side works the same way as yours.
Server logic	Python with the Django framework. Gunicorn is the program that runs Django, the same job Uvicorn does for FastAPI.
Data storage	Most data (users, photo metadata, tags) in PostgreSQL. Run with a main database plus replicas (live copies), and frequent snapshot backups. Later they split data across several databases (sharding).
Speed helpers	Pgbouncer to reuse database connections, Redis for feeds and sessions, Memcached as a cache. You do not need these on day one.
Monitoring	Sentry, an error tracker written in Django, to see crashes in real time.
Hosting	Rented cloud machines (Amazon EC2, Ubuntu Linux), not their own hardware.
Why these choices	Their stated principles: keep it simple, do not reinvent the wheel, prefer proven technology. With three engineers they had no time to run their own infrastructure.

Example B - FastAPI at Netflix, Uber and Microsoft

FastAPI's own documentation quotes engineers from three large companies. Notice what they use it for: specific services, not the whole company website.

Company	What FastAPI does there (per the FastAPI project's quotes)
Netflix	Dispatch, Netflix's open-source crisis management orchestration framework, is built with FastAPI.
Uber	A REST server that other programs query to get predictions from machine-learning models.
Microsoft	An engineer said he planned to use it for his team's machine-learning services.



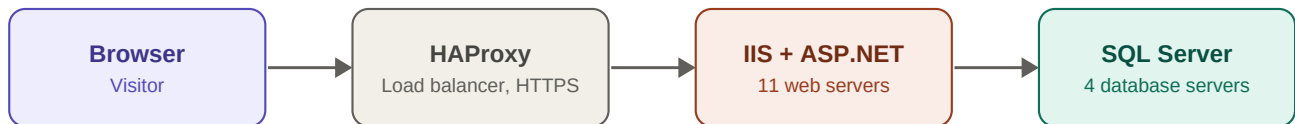
The prediction-service pattern: FastAPI is the front door to Python code.

Why this matters for you

- FastAPI is trusted in production by big teams, mainly for APIs and Python-heavy services (data, machine learning).
- Their roles for it match yours: receive a request, validate it, run Python, answer in JSON.
- I did not verify the databases or frontends of these particular services, so do not copy any layer beyond FastAPI itself.

Example C - Stack Overflow (ASP.NET, SQL Server, a few strong servers)

A very different philosophy that proves a point: you do not need hundreds of servers to run a huge website. Their engineers published the numbers from 2013 and 2016.



Redis (cache) and Elasticsearch (search) sit beside the web tier. Cloudflare sat in front by 2016.

Question	Stack Overflow's answer
What the customer sees	Pages built on the server (server-side templating) and sent as HTML. Not a heavy JavaScript app.
Server logic	C# with Microsoft's ASP.NET MVC on IIS web servers, 11 of them in the 2016 snapshot.
Data storage	Microsoft SQL Server, 4 database servers in 2016. Most queries finish in a few milliseconds because of good indexes.
Speed helpers	Redis cache (2 servers), Elasticsearch for search (3 servers). They moved search from Lucene.NET to Elasticsearch when load grew.
Entry point and HTTPS	HAProxy load balancers spread traffic and handle HTTPS encryption. Nginx did this earlier, then HAProxy took over.
Hosting	Their own machines, with big RAM on the database servers. Their engineers noted that cloud pricing for such large servers would be very expensive.
Why these choices	Few, powerful, well-tuned servers kept the system simple and fast. The database servers ran at low CPU load.

Side-by-side comparison with your plan

Layer	Your plan	Instagram (2011)	Netflix Dispatch / Uber	Stack Overflow (2016)
Language	Python	Python	Python	C#
Web framework	FastAPI	Django + Gunicorn	FastAPI	ASP.NET MVC on IIS
Frontend	HTML, CSS, JS	Native phone app	Not verified	Server-built HTML pages
Database	PostgreSQL	PostgreSQL (replicas, sharding)	Not verified	SQL Server (4)
Cache	None at first	Redis, Memcached	Not verified	Redis (2)
Search	SQL queries	Solr for geo search	Not verified	Elasticsearch (3)
HTTPS and entry	Caddy or Nginx	Not covered in the source	Not verified	HAProxy (+ Cloudflare)
Hosting	One rented VPS	Amazon EC2 (100+ machines)	Not verified	Own servers (25 in 2013)
Monitoring	Logs + uptime monitor	Sentry	Not verified	Dedicated monitoring tools
Team size	You	3 engineers at 14M users	Large teams	Small core team

What to copy now, and what to skip until you need it

Copy now	Skip for now
PostgreSQL as the one database (Instagram's choice)	Sharding and database replicas
Python framework plus a production web server (Gunicorn or Uvicorn)	Redis or Memcached, until pages are measurably slow
An error tracker such as Sentry (it has a free tier)	Elasticsearch or Solr, until SQL search is too slow
Backups from the very start	Load balancers and many servers
HTTPS and a reverse proxy in front of your app	Microservices (splitting into many small services)
Simplicity: use proven tools, do not reinvent the wheel	Kubernetes

Lessons from all three

- **Every company has the same layers you are learning:** something the customer sees, server logic, a database, a safe entry point with HTTPS, and a place it runs. The names differ; the layers do not.
- **Start simple and add only when something hurts.** Instagram began with Django and PostgreSQL and added caches and sharding as users grew. Stack Overflow stayed simple with a few strong machines.
- **Pick one database and learn it well.** Both Instagram and Stack Overflow got very far with a single relational database.
- **FastAPI is a strong choice for APIs and Python-heavy work.** If you later want a site with lots of built-in features (admin panel, user accounts), Django, the framework Instagram used, is also worth learning.
- **Your small site does not need big-company parts.** One server with Docker, PostgreSQL, Caddy and good backups is a genuinely professional starting point.

Sources

- Instagram Engineering, "What Powers Instagram: Hundreds of Instances, Dozens of Technologies": instagram-engineering.com
- High Scalability, "Instagram Architecture Update: What's new with Instagram?": highscalability.com/instagram-architecture-update-whats-new-with-instagram/
- FastAPI documentation home page (user quotes from Netflix, Uber, Microsoft): fastapi.tiangolo.com
- Nick Craver, "Stack Overflow: The Architecture - 2016 Edition": nickcraver.com/blog/2016/02/17/stack-overflow-the-architecture-2016-edition/
- Nick Craver, "What it takes to run Stack Overflow" (2013): nickcraver.com/blog/2013/11/22/what-it-takes-to-run-stack-overflow
- High Scalability, "StackOverflow Update: 560M Pageviews a Month, 25 Servers": highscalability.com/stackoverflow-update-560m-pageviews-a-month-25-servers-and-i/